

mình sắp trong 1 cuộc phỏng vấn senior java cho banking, bạn hãy trả lời các câu hỏi dưới đây 1 cách thật chi tiết, dễ hiểu nhất có thể, đi sâu và thực tế doanh nghiệp, tuyệt đối không trả lời theo kiểu liệt kê, phải trả lời dưới góc độ 1 người có kinh nghiệm thực chiến, các thuật ngữ khó (ví dụ: FaultTolerant, Chunk-oriented processing, Orchestration, ...) thì sẽ note ra ở phía cuối cùng của phần trả lời. dưới đây là các câu hỏi đó:

2. Trong một hệ thống xử lý giao dịch lớn, việc kiểm soát bộ nhớ rất nghiêm ngặt. dùng String, StringBuilder, và StringBuffer như thế nào để tối ưu hiệu năng khi cần ghi

Xem thêm 

Mình sẽ trả lời theo kiểu phỏng vấn thực chiến: mỗi câu có góc nhìn thiết kế, lỗi thường gặp trong banking, ví dụ triển khai Spring/Java, rồi cuối mỗi phần có ghi chú thuật ngữ khó.

Đã suy nghĩ trong giây lát >

Dưới đây là cách mình sẽ trả lời theo góc nhìn **Senior Java trong hệ thống banking**, không trả lời kiểu học thuộc, mà đi theo hướng: tại sao vấn đề nguy hiểm, trong doanh nghiệp xử lý thế nào, trade-off ra sao, và khi phỏng vấn nên nói gì để thể hiện kinh nghiệm thực chiến.

2. Trong hệ thống xử lý giao dịch lớn, kiểm soát bộ nhớ nghiêm ngặt. Dùng String, StringBuilder, StringBuffer như thế nào để tối ưu hiệu năng khi cần ghi log lịch sử giao dịch liên tục?

Trong hệ thống banking, đặc biệt là các hệ thống xử lý giao dịch có TPS cao, việc ghi log lịch sử giao dịch không đơn giản chỉ là nối chuỗi rồi in ra file. Một lỗi rất phổ biến là dùng toán tử + để ghép nhiều chuỗi trong vòng lặp hoặc trong luồng xử lý có tần suất cao. Về mặt cú pháp thì nhìn rất sạch, nhưng phía sau Java sẽ tạo ra nhiều object trung gian, làm tăng áp lực lên heap memory và Garbage Collector. Trong hệ thống giao dịch lớn, chuyện này có thể làm latency tăng bất thường, đặc biệt khi log được ghi liên tục cho audit, reconciliation, transaction trace hoặc fraud monitoring.

Với `String`, mình chỉ dùng cho dữ liệu bất biến, ít thay đổi, hoặc các đoạn text cố định như message template, mã lỗi, trạng thái giao dịch, tên field. `String` là immutable, nghĩa là mỗi lần thay đổi thực chất là tạo object mới. Nếu chỉ có một vài phép nối chuỗi đơn giản, ví dụ build message một lần ngoài vòng lặp, compiler có thể tối ưu giúp bằng `StringBuilder` ngầm. Nhưng trong những luồng như batch xử lý hàng triệu giao dịch, hoặc service ghi log chi tiết cho mỗi bước debit, credit, fee, cashback, reversal, thì tuyệt đối không nên nối chuỗi tùy tiện bằng `+` trong loop.

`StringBuilder` là lựa chọn chính khi cần build nội dung log trong cùng một thread. Ví dụ trong một request chuyển tiền, một thread xử lý request đó sẽ tạo ra một chuỗi log gồm `transactionId`, `fromAccount`, `toAccount`, `amount`, `channel`, `traceId`, `responseCode`, `processingTime`. Lúc này `StringBuilder` phù hợp vì nó mutable, thao tác append không tạo ra nhiều object mới như `String`. Thực tế mình thường khởi tạo

`StringBuilder` với capacity dự đoán trước, ví dụ `new StringBuilder(256)` hoặc `new StringBuilder(512)`, nếu biết log line trung bình khoảng vài trăm ký tự. Việc này giúp tránh việc internal buffer phải resize nhiều lần.

Ví dụ:

```
StringBuilder logBuilder = new StringBuilder(512);

logBuilder.append("txnId=").append(txnId)
    .append("|channel=").append(channel)
    .append("|from=").append(maskAccount(fromAccount))
    .append("|to=").append(maskAccount(toAccount))
    .append("|amount=").append(amount)
    .append("|status=").append(status)
    .append("|durationMs=").append(duration);

log.info(logBuilder.toString());
```

Trong banking, một điểm rất quan trọng là không chỉ tối ưu bộ nhớ mà còn phải đảm bảo không log dữ liệu nhạy cảm. Dù dùng `StringBuilder`, mình vẫn phải mask số tài khoản, số thẻ, CIF, phone, token, OTP, hoặc bất kỳ dữ liệu định danh nào. Việc tối ưu hiệu năng nhưng làm lộ dữ liệu khách hàng là không chấp nhận được.

`StringBuffer` cũng giống `StringBuilder` ở chỗ có thể thay đổi nội dung, nhưng nó synchronized, tức là thread-safe. Vì có khóa đồng bộ nên hiệu năng kém hơn `StringBuilder`. Trong thực tế hiện đại, mình rất ít khi dùng `StringBuffer` để build log trong web application hoặc microservice, vì mỗi request thường được xử lý trong một thread riêng, biến builder cũng là local variable, không chia sẻ giữa các thread. Khi không có chia sẻ object giữa thread, dùng `StringBuffer` là thừa chi phí.

Nếu có nhu cầu ghi log lịch sử giao dịch liên tục ở scale lớn, mình cũng không tự build toàn bộ log bằng chuỗi dài rồi ghi trực tiếp synchronous vào disk. Cách làm thực tế là dùng logging framework như Logback hoặc Log4j2 với async appender, output dạng structured log như JSON, đẩy sang Kafka, ELK, OpenSearch hoặc hệ thống audit riêng. Ở tầng code nghiệp vụ, mình chỉ tạo các field cần thiết, còn format và append ra ngoài nên giao cho framework. Nếu ghi log đồng bộ trong critical path của giao dịch, hệ thống có thể bị chậm vì I/O log, không phải vì xử lý nghiệp vụ.

Một điểm nữa trong phỏng vấn senior nên nói là không lạm dụng log detail ở INFO trong các luồng có TPS cao. Log lịch sử giao dịch phục vụ audit thì cần có, nhưng phải phân loại rõ audit log, application log, debug log, security log. Audit log thường cần bền vững, có traceId, transactionId, requestId, userId đã mask, trạng thái trước/sau, mã lỗi, thời điểm xử lý. Application log thì chỉ nên đủ để điều tra sự cố. Nếu log quá nhiều, chi phí lưu trữ tăng rất mạnh, query chậm, và có thể gây nghẽn I/O.

Tóm lại, mình sẽ dùng `String` cho dữ liệu bất biến và message cố định, dùng `StringBuilder` cho việc build log tạm thời trong phạm vi một thread/request, gần như không dùng `StringBuffer` trừ khi thực sự có object builder bị chia sẻ giữa nhiều thread. Với hệ thống banking lớn, tối ưu đúng không chỉ là chọn class nào, mà còn là kiểm soát object allocation, tránh log synchronous quá nặng, mask dữ liệu nhạy cảm và dùng structured async logging.

Ghi chú thuật ngữ:

`Immutable` là bất biến, object tạo ra rồi thì nội dung không thay đổi.

`Garbage Collector` là cơ chế Java dọn các object không còn được sử dụng.

`Heap memory` là vùng nhớ JVM dùng để chứa object.

`Async logging` là ghi log bất đồng bộ, tránh chặn luồng xử lý chính.

`Structured log` là log có cấu trúc, thường ở dạng JSON, dễ tìm kiếm và phân tích.

3. Tại sao tuyệt đối không được dùng `float` hay `double` để lưu số dư tài khoản hoặc số tiền giao dịch trong Java? Phương án thay thế chính xác là gì?

Trong banking, `float` và `double` gần như là điều cấm kỵ khi lưu số tiền, số dư, phí, lãi suất đã tính ra tiền hoặc bất kỳ giá trị tài chính nào cần độ chính xác tuyệt đối. Lý do là `float` và `double` biểu diễn số theo chuẩn floating-point nhị phân. Có nhiều số thập phân nhìn rất đơn giản với con người, ví dụ `0.1`, `0.2`, nhưng trong hệ nhị phân lại không biểu diễn chính xác được. Kết quả là phép tính có thể sinh ra sai số rất nhỏ, nhưng trong banking, sai một đồng cũng là sai.

Ví dụ kinh điển:

```
double a = 0.1;
double b = 0.2;
System.out.println(a + b);
```

Kết quả có thể là:

```
0.30000000000000004
```



Với hệ thống bình thường, sai số này có thể không quan trọng. Nhưng với tài khoản ngân hàng, lãi suất, phí giao dịch, tất toán tiền gửi, đối soát cuối ngày, sai số tích lũy qua hàng triệu giao dịch có thể gây lệch sổ, lệch GL, lệch báo cáo kế toán. Đây không còn là lỗi kỹ thuật nhỏ, mà là lỗi nghiệp vụ nghiêm trọng.

Phương án thay thế chuẩn trong Java là dùng `BigDecimal` cho các phép tính tiền tệ cần độ chính xác thập phân. `BigDecimal` biểu diễn số theo decimal, có thể kiểm soát scale và rounding mode rõ ràng. Nhưng dùng `BigDecimal` cũng phải đúng cách. Một lỗi rất phổ biến là khởi tạo `BigDecimal` từ `double` :

```
BigDecimal amount = new BigDecimal(0.1);
```

Cách này vẫn sai vì giá trị `0.1` đã bị sai từ lúc nó là `double` . Cách đúng là khởi tạo từ `String` hoặc dùng `BigDecimal.valueOf` :

```
BigDecimal amount1 = new BigDecimal("0.1");
BigDecimal amount2 = BigDecimal.valueOf(0.1);
```

Trong banking, mình thường thấy có hai cách lưu tiền. Ở tầng nghiệp vụ Java, nếu cần tính toán tiền tệ, dùng `BigDecimal` . Ở tầng database, dùng `DECIMAL` hoặc `NUMERIC` , ví dụ `DECIMAL(19, 2)` cho VND nếu chỉ cần đến đơn vị đồng, hoặc scale lớn hơn nếu xử lý ngoại tệ, phí, tỷ giá, lãi suất. Tuyệt đối không dùng kiểu `FLOAT` , `DOUBLE` , `REAL` trong database cho amount.

Ngoài ra, trong một số hệ thống hiệu năng cực cao, người ta có thể lưu amount dưới dạng `long` theo đơn vị nhỏ nhất. Ví dụ với VND, lưu số tiền là đồng thì `100000` nghĩa là một trăm nghìn đồng. Với USD, có thể lưu theo cent, tức là `12345` nghĩa là `123.45 USD`. Cách này rất nhanh và tránh sai số floating-point, nhưng phải cực kỳ nhất quán về

currency minor unit. Vấn đề sẽ phức tạp hơn khi có những đồng tiền không có 2 chữ số thập phân hoặc có quy tắc làm tròn riêng.

Với hệ thống banking thông thường, câu trả lời an toàn nhất là: dùng `BigDecimal` trong Java, dùng `DECIMAL/NUMERIC` trong database, quy định rõ `scale`, `precision`, `rounding mode`, và không bao giờ khởi tạo `BigDecimal` từ `double`. Khi làm các nghiệp vụ như tính lãi, tính phí, chia tỷ lệ, hoàn tiền một phần, hệ thống phải có rule làm tròn rõ ràng, ví dụ `HALF_UP`, `HALF_EVEN`, làm tròn ở từng bước hay làm tròn cuối kỳ. Đây là điểm phỏng vấn rất hay, vì senior không chỉ nói "dùng `BigDecimal`", mà phải nói thêm "dùng `rounding policy` thống nhất theo nghiệp vụ".

Ví dụ:

```
BigDecimal balance = new BigDecimal("1000000.00");
BigDecimal feeRate = new BigDecimal("0.015");

BigDecimal fee = balance.multiply(feeRate)
    .setScale(2, RoundingMode.HALF_UP);
```

Một điểm thực tế nữa là khi compare `BigDecimal`, không nên dùng `equals` nếu chỉ muốn so sánh giá trị số học. Vì `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` trả về `false` do khác `scale`. Trong nghiệp vụ tiền, thường dùng `compareTo`:

```
if (amount.compareTo(BigDecimal.ZERO) <= 0) {
    throw new InvalidAmountException();
}
```

Tóm lại, không dùng `float` và `double` vì chúng có sai số biểu diễn nhị phân. Dùng `BigDecimal` cho tính toán tiền tệ, `DECIMAL/NUMERIC` ở database, hoặc `long` minor unit trong các hệ thống yêu cầu hiệu năng rất cao. Và điều quan trọng nhất là phải thống nhất quy tắc `scale`, `rounding`, `currency` và `compare` trong toàn hệ thống.

Ghi chú thuật ngữ:

Floating-point là kiểu biểu diễn số thực bằng dấu phẩy động, có thể sinh sai số.

`BigDecimal` là class Java dùng để tính toán số thập phân chính xác.

`Scale` là số chữ số sau dấu thập phân.

`Precision` là tổng số chữ số có nghĩa.

`RoundingMode` là quy tắc làm tròn số.

4. Gateway nhận hàng ngàn request truy vấn số dư cùng lúc từ Mobile App. Cấu hình `corePoolSize`, `maxPoolSize`, `queueCapacity` của `ThreadPoolExecutor` dựa trên cơ sở nào để hệ thống không bị nghẽn?

Khi cấu hình thread pool cho Gateway hoặc service xử lý truy vấn số dư, mình không chọn số theo cảm tính kiểu "set `maxPoolSize` thật lớn cho khỏe". Trong thực tế, set thread quá lớn có thể làm hệ thống chết nhanh hơn vì context switching nhiều, memory stack tăng, connection pool bị cạn, downstream bị bắn quá tải và cuối cùng latency tăng dây chuyền.

Đầu tiên phải hiểu bản chất workload. Truy vấn số dư thường là I/O-bound nhiều hơn CPU-bound, vì request đi qua gateway, xác thực token, kiểm tra hạn mức, gọi account service hoặc core banking, lấy dữ liệu từ cache/database/core rồi trả về. Nếu phần lớn thời gian thread đang chờ network/database/core response, số thread có thể lớn hơn số CPU core. Nhưng lớn bao nhiêu phải dựa trên latency trung bình, latency p95/p99, throughput mục tiêu, giới hạn connection pool, giới hạn downstream và SLA.

Một công thức tư duy hay dùng là Little's Law: số request đồng thời xấp xỉ bằng throughput nhân với latency. Ví dụ Gateway cần xử lý 2.000 request/giây, và thời gian xử lý trung bình cho balance inquiry là 100ms, thì số request đang xử lý đồng thời khoảng 200. Nếu p95 latency là 300ms, concurrency có thể lên 600. Thread pool, connection pool và queue phải được sizing dựa trên các con số này, không phải đoán.

`corePoolSize` nên phản ánh mức tải ổn định mà hệ thống thường xuyên xử lý. Nếu hệ thống bình thường có khoảng 100 request đồng thời cho API balance, core pool có thể quanh mức đó, sau khi đã benchmark. `maxPoolSize` là mức chịu tải burst trong thời gian ngắn, nhưng không được vượt quá khả năng của downstream. Nếu account service chỉ có connection pool 200, core banking chỉ cho phép 300 concurrent sessions, thì gateway set 1000 threads là tự phá hệ thống. Gateway phải bảo vệ downstream, không phải chỉ bảo vệ bản thân nó.

`queueCapacity` là điểm rất nhạy. Queue lớn nhìn có vẻ giúp không reject request, nhưng trong banking API online, queue quá lớn thường làm trải nghiệm tệ hơn. Request bị xếp hàng lâu, đến khi được xử lý thì user đã timeout, mobile app retry, hệ thống càng bị dồn thêm traffic. Vì vậy với API realtime như truy vấn số dư, mình thường dùng queue có giới hạn vừa phải, kết hợp timeout, rate limiting, bulkhead và rejection rõ ràng. Thà trả lỗi nhanh như 429 Too Many Requests hoặc lỗi tạm thời có mã nghiệp vụ rõ ràng, còn hơn để request treo 20 giây rồi timeout hàng loạt.

Ví dụ cấu hình ban đầu có thể như sau, nhưng phải nhấn mạnh đây chỉ là ví dụ, thực tế phải load test:

```
@Bean
public ThreadPoolTaskExecutor balanceInquiryExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(100);
    executor.setMaxPoolSize(300);
    executor.setQueueCapacity(500);
    executor.setThreadNamePrefix("balance-inquiry-");
    executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
    executor.initialize();
    return executor;
}
```

Tuy nhiên, trong hệ thống banking lớn, mình sẽ rất cẩn thận với `CallerRunsPolicy`. Policy này khiến thread gọi trực tiếp chạy task khi pool đầy, có thể tạo backpressure tự nhiên. Nhưng ở Gateway, nếu không kiểm soát tốt, nó có thể làm chậm event/request thread chính. Nhiều trường hợp mình thích dùng `AbortPolicy` rồi map sang response lỗi chuẩn để client retry theo chiến lược có kiểm soát.

Ngoài thread pool, còn phải nhìn toàn bộ chuỗi tài nguyên. Nếu service gọi database, connection pool như HikariCP phải được sizing tương thích. Nếu thread pool 300 nhưng DB connection pool chỉ 30, 270 thread sẽ chỉ đứng chờ connection. Nếu gọi core banking qua HTTP/TCP, HTTP client connection pool, read timeout, connect timeout cũng phải đồng bộ. Cấu hình thread pool tách rời khỏi connection pool là lỗi rất thường gặp.

Trong môi trường production, mình không chỉ cấu hình rồi để đó. Mình phải monitor active thread count, queue size, completed task count, rejected count, latency p95/p99, timeout rate, downstream error rate. Khi queue bắt đầu tăng liên tục, đó là tín hiệu hệ thống đang xử lý chậm hơn tốc độ nhận request. Nếu chỉ tăng thread pool mà không xử lý bottleneck downstream, lỗi sẽ chuyển từ nghẽn Gateway sang sập Account Service hoặc Core.

Nếu hệ thống dùng Spring WebFlux hoặc reactive stack, câu chuyện lại khác, vì không nên dùng thread-per-request như truyền thống. Nhưng nếu đang dùng Spring MVC/Tomcat và `ThreadPoolExecutor`, thì nguyên tắc vẫn là sizing dựa trên throughput, latency, CPU, I/O wait, downstream capacity, connection pool và kết quả load test.

Tóm lại, `corePoolSize` dựa vào tải ổn định, `maxPoolSize` dựa vào khả năng chịu burst nhưng không vượt giới hạn downstream, `queueCapacity` dựa vào SLA và khả năng chấp nhận chờ. Với API truy vấn số dư realtime, mình ưu tiên queue có giới hạn, timeout ngắn hợp lý, fail-fast khi quá tải, kết hợp rate limiting và circuit breaker thay vì cố giữ tất cả request trong hàng đợi.

Ghi chú thuật ngữ:

ThreadPoolExecutor là bộ quản lý nhóm thread để xử lý task.

corePoolSize là số thread lõi được giữ trong pool.

maxPoolSize là số thread tối đa pool có thể tạo.

queueCapacity là số task có thể chờ trong hàng đợi.

Backpressure là cơ chế làm chậm hoặc từ chối input khi hệ thống phía sau quá tải.

Little's Law là nguyên lý ước lượng concurrency từ throughput và latency.

p95/p99 latency là thời gian phản hồi mà 95% hoặc 99% request nằm dưới mức đó.

5. Khi dùng @Transactional trong Spring Boot cho luồng chuyển tiền, rollback khác nhau thế nào giữa RuntimeException và CheckedException ? Làm sao customize lại?

Trong Spring, mặc định @Transactional sẽ rollback khi method ném ra

RuntimeException hoặc Error, nhưng sẽ không rollback nếu ném ra

CheckedException, trừ khi mình cấu hình rõ. Đây là điểm rất quan trọng trong luồng chuyển tiền, vì nếu hiểu sai có thể dẫn đến trạng thái dữ liệu cực kỳ nguy hiểm.

Ví dụ một luồng chuyển tiền nội bộ thường có các bước như kiểm tra số dư, ghi debit tài khoản nguồn, ghi credit tài khoản đích, ghi transaction history, ghi ledger, publish event. Nếu một lỗi xảy ra giữa chừng, toàn bộ transaction database phải rollback để không có tình trạng tài khoản nguồn bị trừ nhưng tài khoản đích chưa được cộng.

Với RuntimeException, Spring sẽ rollback mặc định:

```
@Transactional
public void transfer(String from, String to, BigDecimal amount) {
    debit(from, amount);
    credit(to, amount);

    if (someCondition) {
        throw new IllegalStateException("Transfer failed");
    }
}
```

Trong ví dụ này, IllegalStateException là RuntimeException, nên Spring đánh dấu transaction rollback-only và rollback khi method thoát ra.

Nhưng nếu mình ném một CheckedException như sau:


```
@Transactional
public void transfer(String from, String to, BigDecimal amount) throws TransferException {
    debit(from, amount);
    credit(to, amount);

    throw new TransferException("Transfer failed");
}
```

Nếu `TransferException` extends `Exception` chứ không extends `RuntimeException`, Spring mặc định sẽ không rollback. Điều này làm nhiều developer bất ngờ. Lý do lịch sử là checked exception thường được xem như exception nghiệp vụ có thể dự đoán và xử lý được, không nhất thiết là lỗi hệ thống. Nhưng trong banking, nhiều checked exception vẫn phải rollback, ví dụ lỗi gọi core, lỗi validate nghiệp vụ sau khi đã ghi trạng thái tạm, lỗi đối soát nội bộ.

Để customize, mình dùng `rollbackFor`:

```
@Transactional(rollbackFor = TransferException.class)
public void transfer(String from, String to, BigDecimal amount) throws TransferException {
    debit(from, amount);
    credit(to, amount);

    throw new TransferException("Transfer failed");
}
```

Nếu có nhiều exception cần rollback:

```
@Transactional(rollbackFor = {TransferException.class, CoreBankingException.class})
```

Ngược lại, nếu có một số `RuntimeException` nhưng không muốn rollback, có thể dùng `noRollbackFor`:

```
@Transactional(noRollbackFor = NotificationException.class)
public void transfer(...) {
    debit(...);
    credit(...);
    sendNotification(...);
}
```

Tuy nhiên, trong thực tế banking, mình không muốn notification failure làm rollback chuyển tiền nếu giao dịch tiền đã thành công. Thường phần gửi notification nên tách khỏi transaction chính, dùng event/outbox để xử lý sau. Không nên để việc gửi SMS/push notification nằm trong cùng transaction database của debit/credit.

Một điểm senior nên nói thêm là `@Transactional` chỉ có hiệu lực khi method được gọi thông qua Spring proxy. Nếu trong cùng một class, method A gọi trực tiếp method B có `@Transactional`, thì transaction có thể không được áp dụng như mong muốn do self-invocation. Đây là lỗi rất phổ biến trong dự án Spring. Với luồng chuyển tiền, mình thường đặt transaction boundary ở service layer public method, không đặt rải rác ở private method.

Ngoài ra, rollback của `@Transactional` chỉ rollback tài nguyên nằm trong transaction manager, thường là database transaction. Nó không tự rollback việc đã gọi API bên ngoài, đã gửi Kafka message trực tiếp, đã gửi SMS, hoặc đã gọi sang core banking nếu core đã xử lý. Vì vậy trong kiến trúc microservices banking, phải kết hợp thêm Outbox Pattern, Saga, compensation hoặc trạng thái giao dịch rõ ràng. Không nên nghĩ `@Transactional` là cây đũa thần rollback mọi thứ.

Tóm lại, mặc định Spring rollback với `RuntimeException` và `Error`, không rollback với `CheckedException`. Muốn thay đổi thì dùng `rollbackFor` hoặc `noRollbackFor`. Trong luồng chuyển tiền, mình sẽ định nghĩa rõ exception nào là lỗi kỹ thuật phải rollback, exception nào là lỗi nghiệp vụ có thể commit trạng thái rejected, và tuyệt đối không để side-effect bên ngoài nằm mù mờ trong transaction database.

Ghi chú thuật ngữ:

`RuntimeException` là exception không bắt buộc phải khai báo hoặc catch.

`CheckedException` là exception bắt buộc khai báo hoặc xử lý.

Rollback là hoàn tác các thay đổi trong transaction.

Transaction boundary là phạm vi bắt đầu và kết thúc transaction.

Spring proxy là object trung gian Spring tạo ra để áp dụng các cơ chế như transaction, security, logging.

Self-invocation là việc một method trong cùng class gọi trực tiếp method khác của chính class đó, có thể làm mất tác dụng proxy.

6. Service Thẻ gọi Service Tài khoản qua FeignClient nhưng Service Tài khoản phản hồi rất chậm hoặc timeout. Xử lý thế nào để không làm sập dây chuyền toàn hệ thống? Đã từng cấu hình thông số Circuit Breaker như Resilience4j thế nào?

Trong microservices banking, khi Service Thẻ gọi Service Tài khoản qua FeignClient, điều nguy hiểm nhất không chỉ là Service Tài khoản lỗi, mà là nó chậm. Lỗi nhanh còn dễ xử lý. Chậm mới nguy hiểm vì thread bị giữ lại, connection bị chiếm, queue tăng, rồi Service Thẻ cũng chậm theo. Mobile Gateway tiếp tục chờ Service Thẻ, client retry, traffic tăng lên, và cuối cùng cả chuỗi bị nghẽn. Đây là kiểu cascading failure rất hay gặp.

Cách xử lý đầu tiên là phải có timeout rõ ràng. Không bao giờ để FeignClient dùng timeout quá dài hoặc default không kiểm soát. Mình thường tách `connectTimeout` và `readTimeout`. `connectTimeout` là thời gian chờ thiết lập kết nối, thường ngắn hơn. `readTimeout` là thời gian chờ response sau khi đã kết nối. Với API online như truy vấn thông tin tài khoản để phục vụ thẻ, timeout thường phải nhỏ, ví dụ vài trăm ms đến vài giây tùy SLA nội bộ. Không thể để một request treo 30 giây trong hệ thống realtime.

Ví dụ cấu hình Feign:

```
spring:
  cloud:
    openfeign:
      client:
        config:
          account-service:
            connectTimeout: 500
            readTimeout: 1500
```

Sau timeout, cần có Circuit Breaker để nếu Service Tài khoản đang lỗi hoặc quá chậm, Service Thẻ không tiếp tục bắn request vô ích. Circuit Breaker hoạt động giống cầu dao điện. Khi tỷ lệ lỗi hoặc slow call vượt ngưỡng, circuit chuyển sang OPEN, các request sau fail-fast ngay, không gọi downstream nữa. Sau một thời gian, circuit chuyển sang HALF_OPEN để thử một số request. Nếu downstream hồi phục, circuit đóng lại.

Trong Resilience4j, những thông số mình hay cấu hình thực tế gồm `failureRateThreshold`, `slowCallRateThreshold`, `slowCallDurationThreshold`, `slidingWindowSize`, `minimumNumberOfCalls`, `waitDurationInOpenState`, `permittedNumberOfCallsInHalfOpenState`, ngoài ra có thể kết hợp `TimeLimiter`, `Retry` và `Bulkhead`.

Ví dụ:

```
resilience4j:
  circuitbreaker:
```

```

instances:
  accountService:
    slidingWindowType: COUNT_BASED
    slidingWindowSize: 100
    minimumNumberOfCalls: 50
    failureRateThreshold: 50
    slowCallRateThreshold: 60
    slowCallDurationThreshold: 2s
    waitDurationInOpenState: 10s
    permittedNumberOfCallsInHalfOpenState: 10
    automaticTransitionFromOpenToHalfOpenEnabled: true

timelimiter:
  instances:
    accountService:
      timeoutDuration: 1500ms

bulkhead:
  instances:
    accountService:
      maxConcurrentCalls: 100
      maxWaitDuration: 0

```

Nhưng trong banking, fallback không được làm bừa. Ví dụ nếu Service Thẻ gọi Service Tài khoản để kiểm tra số dư trước khi duyệt giao dịch tài chính, không thể fallback bằng cách “cho qua” hoặc dùng dữ liệu cache cũ nếu điều đó có thể gây thất thoát tiền. Với các API chỉ hiển thị thông tin, có thể fallback trả dữ liệu cache kèm cảnh báo “dữ liệu có thể chưa cập nhật”. Nhưng với giao dịch tiền thật, fallback an toàn thường là từ chối tạm thời, chuyển trạng thái pending, hoặc yêu cầu retry sau. Đây là điểm rất quan trọng để thể hiện tư duy banking: availability không được đánh đổi bằng sai lệch tiền.

Mình cũng thường kết hợp Bulkhead để giới hạn số concurrent call sang Account Service. Nếu không có Bulkhead, một downstream chậm có thể chiếm hết thread của Service Thẻ, khiến các chức năng khác cũng chết theo. Bulkhead giúp cô lập lỗi, ví dụ chỉ cho tối đa 100 request đang gọi Account Service. Phần còn lại fail-fast hoặc queue rất ngắn.

Retry cũng phải dùng rất cẩn thận. Với request đọc như lấy thông tin tài khoản, retry có thể chấp nhận nếu timeout ngắn và số lần retry ít. Nhưng với request ghi như debit, credit, block amount, capture payment, retry mù quáng có thể gây double posting nếu downstream không idempotent. Trong banking, retry phải đi cùng idempotency key, transaction reference và trạng thái giao dịch rõ ràng.

Một điểm thực tế nữa là phải monitor circuit breaker state. Khi circuit open, không chỉ log rồi thôi, mà cần alert, dashboard theo service, theo endpoint, theo error code. Nếu không, đội vận hành không biết hệ thống đang tự bảo vệ hay đang mất kết nối thật. Các

metric như failure rate, slow call rate, timeout count, rejected by bulkhead, fallback count rất quan trọng.

Tóm lại, mình sẽ xử lý bằng timeout ngắn, circuit breaker, time limiter, bulkhead, fallback an toàn theo nghiệp vụ, retry có kiểm soát và observability đầy đủ. Với banking, fallback không phải lúc nào cũng là trả dữ liệu thay thế; nhiều khi fallback đúng là fail-fast và giữ trạng thái an toàn cho tiền của khách hàng.

Ghi chú thuật ngữ:

`FeignClient` là client khai báo trong Spring để gọi HTTP service khác.

`Circuit Breaker` là cơ chế ngắt tạm thời lời gọi đến service lỗi/chậm để tránh sập dây chuyền.

`Fallback` là logic thay thế khi lời gọi chính thất bại.

`Cascading failure` là lỗi lan truyền từ một service sang nhiều service khác.

`Bulkhead` là cơ chế cô lập tài nguyên để một lỗi không chiếm hết toàn hệ thống.

`TimeLimiter` là cơ chế giới hạn thời gian chờ xử lý.

`HALF_OPEN` là trạng thái thử lại có kiểm soát sau khi circuit đã mở.

7. Nếu cần khởi tạo một kết nối bảo mật đến Core ngay khi Spring Boot start thành công, trước khi nhận request, xử lý ở giai đoạn nào trong Lifecycle của Bean hoặc dùng cách nào?

Trong thực tế banking, kết nối đến Core không giống kết nối HTTP thông thường. Có thể đó là kết nối TCP socket, MQ, ISO8583, SOAP over mutual TLS, VPN private endpoint, HSM, hoặc một session cần handshake bảo mật. Nếu ứng dụng start lên nhưng chưa kết nối được Core mà đã nhận request, request đầu tiên có thể fail hoặc gây timeout hàng loạt. Vì vậy có những trường hợp mình muốn khởi tạo kết nối ngay khi app start và chỉ mark service ready khi kết nối Core thành công.

Có nhiều cách xử lý trong lifecycle Spring, nhưng phải chọn đúng thời điểm. Nếu chỉ cần init sau khi bean được tạo và dependency đã inject xong, có thể dùng `@PostConstruct`. Cách này đơn giản, nhưng có nhược điểm là chạy khá sớm trong lifecycle. Nếu quá trình init phụ thuộc vào toàn bộ application context, event listener hoặc các bean khác đã sẵn sàng, `@PostConstruct` có thể chưa phải lựa chọn tốt nhất.

Ví dụ:

```
@Component
public class CoreConnectionManager {
```

```

private final CoreClient coreClient;

public CoreConnectionManager(CoreClient coreClient) {
    this.coreClient = coreClient;
}

@PostConstruct
public void init() {
    coreClient.connectSecurely();
}
}

```

Nếu muốn chạy sau khi Spring context đã refresh xong, mình thường dùng `ApplicationRunner` hoặc `CommandLineRunner`. Đây là cách rất phổ biến để thực hiện logic startup như warm-up cache, verify connection, đăng ký service, preload config hoặc mở session đến core.

```

@Component
public class CoreStartupRunner implements ApplicationRunner {

    private final CoreConnectionManager coreConnectionManager;

    public CoreStartupRunner(CoreConnectionManager coreConnectionManager) {
        this.coreConnectionManager = coreConnectionManager;
    }

    @Override
    public void run(ApplicationArguments args) {
        coreConnectionManager.connectAndHandshake();
    }
}

```

Nếu cần lắng nghe thời điểm app đã sẵn sàng hơn nữa, có thể dùng event `ApplicationReadyEvent`. Event này xảy ra khi application đã sẵn sàng phục vụ request theo quan điểm Spring Boot. Tuy nhiên, nếu mình cần đảm bảo kết nối Core xong rồi mới nhận traffic, thì chỉ lắng nghe `ApplicationReadyEvent` chưa đủ nếu load balancer đã route traffic vào app. Trong môi trường Kubernetes, cách chuẩn hơn là dùng readiness probe. Ứng dụng có thể start, nhưng readiness vẫn trả `DOWN` cho đến khi Core connection initialized thành công. Khi readiness `UP`, traffic mới được đưa vào pod.

Ví dụ tư duy thực tế là không nên block startup vô hạn. Nếu Core chưa kết nối được, mình phải quyết định rõ: fail startup để container restart, hay start app nhưng readiness down và retry nền. Với banking, nếu service không thể hoạt động nếu không có Core, mình thường chọn fail-fast hoặc readiness down. Không nên để service nhận request rồi trả lỗi timeout lung tung.

Có thể kết hợp với Spring Boot Actuator HealthIndicator:

```
@Component
public class CoreHealthIndicator implements HealthIndicator {

    private final CoreConnectionManager manager;

    public CoreHealthIndicator(CoreConnectionManager manager) {
        this.manager = manager;
    }

    @Override
    public Health health() {
        if (manager.isConnected()) {
            return Health.up().build();
        }
        return Health.down()
            .withDetail("core", "Secure connection is not established")
            .build();
    }
}
```

Trong lifecycle bean, nếu muốn quản lý start/stop rõ ràng hơn, có thể implement `SmartLifecycle`. Cách này phù hợp với các component có vòng đời riêng như connection manager, message listener, socket client. `SmartLifecycle` cho phép kiểm soát auto startup, phase start/stop, và graceful shutdown. Với kết nối Core qua TCP/MQ lâu dài, đây là cách khá sạch.

Tóm lại, nếu init đơn giản sau inject thì dùng `@PostConstruct`. Nếu muốn chạy sau khi context sẵn sàng thì dùng `ApplicationRunner` hoặc `CommandLineRunner`. Nếu muốn chạy khi app ready thì dùng `ApplicationReadyEvent`. Nếu cần quản lý vòng đời kết nối dài hạn thì dùng `SmartLifecycle`. Trong production banking, mình sẽ kết hợp thêm readiness probe/health check để đảm bảo app chỉ nhận request khi kết nối bảo mật đến Core đã sẵn sàng.

Ghi chú thuật ngữ:

`Bean Lifecycle` là vòng đời tạo, khởi tạo, sử dụng và hủy bean trong Spring.

`@PostConstruct` là method chạy sau khi bean được tạo và inject dependency xong.

`ApplicationRunner` là interface chạy logic sau khi Spring Boot application start.

`ApplicationReadyEvent` là event báo ứng dụng đã sẵn sàng.

`Readiness probe` là cơ chế kiểm tra service đã sẵn sàng nhận traffic hay chưa.

`SmartLifecycle` là interface Spring dùng để quản lý start/stop của component có vòng đời riêng.

8. Điều gì xảy ra nếu Singleton Bean inject Prototype Bean trong Spring? Làm thế nào để mỗi request vào Singleton Bean đều nhận instance mới của Prototype Bean?

Trong Spring, Singleton Bean là mặc định. Nghĩa là trong application context chỉ có một instance của bean đó. Prototype Bean thì khác, mỗi lần yêu cầu Spring tạo bean, Spring sẽ tạo một instance mới. Nhưng vấn đề nằm ở chỗ: nếu một Singleton Bean inject trực tiếp một Prototype Bean qua constructor hoặc field, thì Prototype Bean chỉ được tạo một lần tại thời điểm Singleton Bean được khởi tạo. Sau đó Singleton giữ mãi reference đó. Kết quả là dù bean kia khai báo scope prototype, trong thực tế Singleton vẫn dùng cùng một instance cho mọi request.

Ví dụ:

```
@Component
public class PaymentService {

    private final TransactionContext transactionContext;

    public PaymentService(TransactionContext transactionContext) {
        this.transactionContext = transactionContext;
    }

    public void process() {
        transactionContext.setTraceId(UUID.randomUUID().toString());
    }
}

@Component
@Scope("prototype")
public class TransactionContext {
    private String traceId;
}
```

Nhìn thì tưởng mỗi lần dùng TransactionContext sẽ là instance mới, nhưng thực tế PaymentService là singleton, được tạo một lần, và TransactionContext được inject một lần vào constructor. Nếu TransactionContext có state như traceId, userId, amount tạm, thì rất nguy hiểm vì nhiều request có thể dùng chung state, gây lẫn dữ liệu giữa các giao dịch.

Cách xử lý phổ biến là inject ObjectProvider hoặc ObjectFactory, để mỗi lần cần dùng thì gọi getObject(). Khi đó Spring sẽ tạo prototype mới cho mỗi lần gọi.


```

@Component
public class PaymentService {

    private final ObjectProvider<TransactionContext> contextProvider;

    public PaymentService(ObjectProvider<TransactionContext> contextProvider) {
        this.contextProvider = contextProvider;
    }

    public void process() {
        TransactionContext context = contextProvider.getObject();
        context.setTraceId(UUID.randomUUID().toString());

        // xử lý giao dịch với context riêng
    }
}

```

Một cách khác là dùng `@Lookup`. Spring sẽ override method để trả về prototype bean mới mỗi lần gọi. Cách này ít dùng hơn trong dự án hiện đại vì hơi magic, khó đọc hơn `ObjectProvider`.

```

@Component
public abstract class PaymentService {

    public void process() {
        TransactionContext context = createContext();
    }

    @Lookup
    protected abstract TransactionContext createContext();
}

```

Ngoài ra có thể dùng `scoped proxy`:

```

@Component
@Scope(value = "prototype", proxyMode = ScopedProxyMode.TARGET_CLASS)
public class TransactionContext {
}

```

Nhưng với `prototype`, mình thường thích `ObjectProvider` vì rõ ràng. `Scoped proxy` hay dùng hơn với `request scope/session scope` trong web application, ví dụ mỗi HTTP request có một `request-scoped bean`.

Tuy nhiên, trong hệ thống banking, mình sẽ đặt câu hỏi ngược lại: có thật sự cần Prototype Bean có state không? Trong thiết kế service tốt, các Spring service nên stateless. Những dữ liệu theo request như transactionId, requestId, userId, amount, channel nên nằm trong method local variable, DTO, command object hoặc context object được tạo thủ công, không nhất thiết phải là Spring bean. Việc đưa state của giao dịch vào Spring bean rất dễ gây bug concurrency.

Tóm lại, Singleton inject trực tiếp Prototype thì prototype chỉ được tạo một lần tại thời điểm singleton được tạo. Muốn mỗi request có instance mới thì dùng `ObjectProvider`, `ObjectFactory`, `@Lookup` hoặc `scoped proxy`. Nhưng trong banking, tốt nhất là service singleton nên stateless, còn dữ liệu giao dịch nên nằm trong object được tạo theo request, không chia sẻ giữa các thread.

Ghi chú thuật ngữ:

Singleton Bean là bean chỉ có một instance trong Spring container.

Prototype Bean là bean được tạo mới mỗi lần được yêu cầu từ Spring container.

`ObjectProvider` là cơ chế lấy bean một cách lazy/dynamic từ Spring.

`Scoped proxy` là proxy đại diện cho bean có scope đặc biệt.

Stateless service là service không giữ state riêng của từng request trong field.

9. Tránh khách hàng bị trừ tiền hai lần khi bấm thanh toán liên tiếp do mạng lag. Thiết kế Idempotency ở tầng API như thế nào?

Trong thanh toán, idempotency là bắt buộc. Vấn đề không chỉ là user bấm nút hai lần. Có rất nhiều nguyên nhân tạo duplicate request: mobile app retry vì timeout, gateway retry, network chậm chờn, client không nhận được response nên gửi lại, partner gửi callback nhiều lần, hoặc message queue redelivery. Nếu API không idempotent, khách hàng có thể bị trừ tiền hai lần cho cùng một ý định thanh toán.

Cách thiết kế thực tế là mỗi payment intent phải có một khóa định danh duy nhất, thường gọi là `Idempotency-Key`, `requestId`, `clientTransactionId`, `orderId` hoặc `paymentReference`. Client hoặc Gateway tạo key này cho một ý định thanh toán. Nếu user bấm lại cùng một giao dịch, request gửi lên phải giữ cùng idempotency key. Server dùng key đó để nhận biết đây là request trùng, không tạo giao dịch mới.

Ví dụ API:

```
POST /api/payments
```

```
Idempotency-Key: 8f4e4b8c-6c1e-4a6e-9b16-9f72d82d91a1
```

Body:

```
{
  "accountNo": "123456789",
  "merchantId": "M001",
  "amount": "100000",
  "currency": "VND",
  "orderId": "ORDER_20260529_0001"
}
```

Ở server, mình tạo một bảng hoặc store riêng cho idempotency:

```
idempotency_key
request_hash
status
transaction_id
response_body
created_at
expired_at
```



Khi request đầu tiên vào, server insert key với trạng thái `PROCESSING`. Insert này phải có unique constraint trên `idempotency_key`, hoặc tốt hơn là unique theo cặp `client_id + idempotency_key` để tránh key của client này đụng client khác. Nếu insert thành công, request được xử lý. Khi giao dịch hoàn tất, record idempotency được cập nhật thành `SUCCESS` hoặc `FAILED`, kèm transaction id và response chuẩn.

Nếu request thứ hai đến với cùng idempotency key trong lúc request đầu tiên đang xử lý, hệ thống không được xử lý debit lần nữa. Tùy thiết kế, có thể trả về trạng thái `PROCESSING`, hoặc chờ ngắn để lấy kết quả request đầu tiên, hoặc trả response "giao dịch đang được xử lý, vui lòng kiểm tra trạng thái". Nếu request thứ hai đến sau khi request đầu tiên thành công, server trả lại chính response cũ, không thực hiện lại debit.

Điểm quan trọng là phải lưu thêm `request_hash`. Vì nếu cùng một idempotency key nhưng body khác, đó là request không hợp lệ. Ví dụ lần đầu amount là 100.000 VND, lần sau dùng cùng key nhưng amount là 200.000 VND, server phải reject với lỗi kiểu "Idempotency key reused with different request payload". Nếu không kiểm tra request hash, có thể xảy ra lỗi bảo mật hoặc sai nghiệp vụ.

Pseudo-flow:

```
@Transactional
public PaymentResponse createPayment(PaymentRequest request, String idempotencyKey)
```

```

String requestHash = hash(request);

Optional<IdempotencyRecord> existing = repository.findByKey(idempotencyKey);

if (existing.isPresent()) {
    IdempotencyRecord record = existing.get();

    if (!record.getRequestHash().equals(requestHash)) {
        throw new InvalidIdempotencyKeyException();
    }

    if (record.isCompleted()) {
        return record.getSavedResponse();
    }

    return PaymentResponse.processing(record.getTransactionId());
}

repository.insertProcessing(idempotencyKey, requestHash);

PaymentResult result = paymentProcessor.process(request);

repository.markCompleted(idempotencyKey, result.getTransactionId(), result.getResponse());

return result.getResponse();
}

```

Trong production, đoạn find rồi insert có thể bị race condition nếu hai request đến cùng lúc. Vì vậy không chỉ check bằng code, mà bắt buộc phải có unique constraint ở database hoặc atomic operation trong Redis như SET NX. Với giao dịch tiền, mình thường vẫn cần database constraint vì nó bền vững và audit tốt hơn. Redis có thể dùng để chặn nhanh ở tầng ngoài, nhưng source of truth nên nằm ở database/payment ledger.

Một chi tiết rất quan trọng là idempotency không thay thế transaction consistency. Bên trong payment processor vẫn phải có transaction boundary, lock tài khoản, kiểm tra số dư, ghi ledger, cập nhật trạng thái giao dịch. Idempotency chỉ đảm bảo cùng một ý định thanh toán không bị thực hiện nhiều lần.

Thời gian sống của idempotency key cũng phải được xác định. Với API thanh toán, có thể lưu vài giờ, vài ngày hoặc theo thời gian đối soát, tùy quy định. Trong banking, mình thường không xóa sớm dữ liệu giao dịch, nhưng bảng idempotency có thể archive sau một thời gian. Nếu tích hợp đối tác, key thường gắn với partnerCode + requestId + orderId.

Tóm lại, thiết kế idempotency tốt ở tầng API gồm: client gửi idempotency key duy nhất cho một ý định giao dịch, server lưu key với unique constraint, kiểm tra request hash, lưu

trạng thái xử lý và response, trả lại response cũ cho request trùng, không debit lại. Với payment, mọi thứ phải được thiết kế theo hướng "retry-safe".

Ghi chú thuật ngữ:

Idempotency là tính chất gọi một lần hay nhiều lần cùng một request vẫn cho cùng một kết quả, không tạo side-effect lặp.

Idempotency-Key là khóa định danh một ý định giao dịch duy nhất.

Request hash là mã băm của request body dùng để phát hiện cùng key nhưng nội dung khác.

Unique constraint là ràng buộc database đảm bảo một giá trị không bị trùng.

Retry-safe là thiết kế cho phép retry mà không gây tác dụng phụ sai.

10. Tích hợp API với ví điện tử/cổng thanh toán: dùng gì để xác thực và bảo vệ dữ liệu? Phân biệt JWT, chữ ký số RSA và mTLS?

Khi tích hợp với đối tác thứ ba như ví điện tử, cổng thanh toán, trung gian thanh toán hoặc biller, mình thường phải giải quyết ba vấn đề khác nhau: xác định bên gọi là ai, đảm bảo dữ liệu không bị sửa trên đường truyền, và đảm bảo kết nối được mã hóa an toàn.

Nhiều bạn hay nhầm JWT, RSA signature và mTLS là một thứ, nhưng thực tế vai trò của chúng khác nhau.

JWT thường dùng để xác thực và truyền claims. Ví dụ đối tác gọi API của ngân hàng, họ gửi `Authorization: Bearer <token>`. Token này có thể chứa `partnerId`, `scope`, `channel`, thời hạn hết hạn, quyền được gọi API nào. Nếu JWT được ký bởi authorization server đáng tin cậy, server nhận có thể verify chữ ký token và biết token chưa bị sửa. JWT phù hợp với bài toán authentication và authorization ở tầng application.

Nhưng JWT không đủ để đảm bảo toàn vẹn từng payload giao dịch theo chuẩn đối soát tài chính. Ví dụ partner gửi yêu cầu thanh toán 1.000.000 VND, mình cần đảm bảo chính nội dung `amount`, `orderId`, `accountNo`, `timestamp`, `nonce` không bị sửa. Ở đây thường dùng chữ ký số RSA hoặc HMAC signature trên canonical request. Partner lấy một chuỗi chuẩn hóa từ `method`, `path`, `timestamp`, `nonce`, `body hash`, rồi ký bằng private key. Ngân hàng dùng public key của partner để verify. Nếu body bị thay đổi dù chỉ một ký tự, chữ ký không còn hợp lệ.

Ví dụ header có thể như sau:

```
X-Partner-Id: WALLET_A
X-Timestamp: 2026-05-29T10:15:30+07:00
X-Nonce: b7b8d4...
```

```
X-Signature: base64-rsa-signature  
Authorization: Bearer eyJ...
```

Trong mô hình này, JWT nói "caller này là ai và có quyền gì", còn RSA signature nói "nội dung request này đúng là do caller ký và chưa bị sửa". Với giao dịch tiền, signature payload cực kỳ quan trọng vì nó phục vụ cả security lẫn non-repudiation ở mức nhất định, tức là đối tác khó phủ nhận họ đã gửi request nếu private key được quản lý đúng.

mTLS lại nằm ở tầng transport. Bình thường HTTPS chỉ xác thực server với client, tức là client biết mình đang nói chuyện với đúng server. Với mTLS, cả client cũng phải trình certificate cho server. Server kiểm tra certificate đó có thuộc partner được tin cậy không. Điều này giúp đảm bảo chỉ hệ thống có certificate hợp lệ mới mở được kết nối. mTLS rất hay dùng trong kết nối B2B banking, đặc biệt qua private network, leased line, VPN hoặc API Gateway.

Tuy nhiên, mTLS không thay thế hoàn toàn JWT hay payload signature. mTLS bảo vệ kênh truyền và xác thực client ở tầng kết nối, nhưng khi request đã đi qua gateway, load balancer, service mesh, hoặc nhiều hop nội bộ, application vẫn cần biết partner nào, quyền gì, request nào, trace nào. Payload signature vẫn cần nếu muốn đảm bảo body không bị sửa ở tầng trung gian hoặc để đối soát/chứng minh về sau.

Trong thực tế mình thường thấy mô hình an toàn gồm HTTPS/mTLS cho transport security, JWT hoặc OAuth2 client credentials cho authentication/authorization, RSA/HMAC signature cho request integrity, thêm timestamp và nonce để chống replay attack. Với dữ liệu nhạy cảm, nếu yêu cầu cao hơn, có thể mã hóa một số field ở application level, ví dụ encrypt số tài khoản hoặc thông tin định danh bằng public key của bên nhận. Nhưng không nên tự chế crypto; phải theo chuẩn thống nhất với security team.

Replay attack là một rủi ro thực tế. Kẻ xấu có thể lấy lại một request hợp lệ và gửi lại nhiều lần. Vì vậy signature thường phải ký cả timestamp và nonce. Server kiểm tra timestamp trong cửa sổ hợp lệ, ví dụ 5 phút, và nonce chưa từng được dùng. Với payment, requestId/orderId cũng phải unique để chống duplicate ở tầng nghiệp vụ.

Tóm lại, JWT dùng để xác thực danh tính và quyền ở tầng application, RSA signature dùng để đảm bảo toàn vẹn payload và xác nhận bên ký, còn mTLS dùng để xác thực hai chiều và mã hóa ở tầng transport. Trong banking integration, mình thường dùng kết hợp chứ không chọn một trong ba.

Ghi chú thuật ngữ:

JWT là token chứa claims, thường dùng cho authentication và authorization.

RSA signature là chữ ký số dùng private key để ký và public key để verify.

mTLS là mutual TLS, cơ chế TLS xác thực cả client và server bằng certificate.

Payload integrity là đảm bảo nội dung request không bị thay đổi.

Replay attack là tấn công gửi lại request hợp lệ cũ để thực hiện lại hành động.

Nonce là giá trị ngẫu nhiên dùng một lần để chống replay.

Canonical request là chuỗi request đã được chuẩn hóa trước khi ký.

11. Spring Batch quét và tính lãi suất toàn bộ tài khoản tiết kiệm cuối ngày. Nếu vài bản ghi lỗi dữ liệu, làm sao để job không chết giữa chừng mà vẫn log lại tài khoản lỗi?

Trong batch banking cuối ngày, việc có một vài bản ghi lỗi dữ liệu là chuyện có thể xảy ra. Ví dụ tài khoản thiếu kỳ hạn, interest rate null, ngày mở sổ sai, trạng thái tài khoản không hợp lệ, hoặc dữ liệu migration từ hệ thống cũ chưa sạch. Nếu batch tính lãi cho vài triệu tài khoản mà gặp một record lỗi rồi chết toàn bộ job, vận hành cuối ngày sẽ rất căng. Vì vậy Spring Batch phải được thiết kế theo hướng fault-tolerant, xử lý được lỗi cục bộ, log lại record lỗi, và tiếp tục với những record hợp lệ nếu nghiệp vụ cho phép.

Với Spring Batch, cách làm phổ biến là dùng chunk-oriented processing. Job đọc một chunk, ví dụ 500 hoặc 1000 tài khoản, xử lý từng item, rồi ghi kết quả. Nếu một item lỗi do dữ liệu không hợp lệ, mình có thể cấu hình skip exception đó, tăng skip count, log record lỗi vào bảng riêng, và tiếp tục xử lý item tiếp theo. Nhưng không phải lỗi nào cũng được skip. Lỗi dữ liệu của một tài khoản có thể skip. Lỗi database down, mất kết nối core, sai công thức tính lãi toàn hệ thống thì không được skip, phải fail job để tránh tính sai hàng loạt.

Ví dụ cấu hình:

```
@Bean
public Step interestCalculationStep(
    JobRepository jobRepository,
    PlatformTransactionManager transactionManager,
    ItemReader<SavingAccount> reader,
    ItemProcessor<SavingAccount, InterestPosting> processor,
    ItemWriter<InterestPosting> writer,
    SkipListener<SavingAccount, InterestPosting> skipListener
) {
    return new StepBuilder("interestCalculationStep", jobRepository)
        .<SavingAccount, InterestPosting>chunk(500, transactionManager)
        .reader(reader)
        .processor(processor)
        .writer(writer)
        .faultTolerant()
        .skip(InvalidAccountDataException.class)
}
```

```

        .skipLimit(1000)
        .listener(skipListener)
        .build();
    }

```

Ở đây, `faultTolerant()` cho phép step xử lý các tình huống lỗi có kiểm soát. `skip()` nói rằng exception nào được phép bỏ qua. `skipLimit()` giới hạn số record lỗi tối đa. Đây là điểm rất quan trọng trong banking. Nếu chỉ 10 tài khoản lỗi trong 5 triệu tài khoản, có thể skip và log. Nhưng nếu 100.000 tài khoản lỗi, có thể là lỗi công thức, lỗi dữ liệu master hoặc lỗi release, lúc đó job phải dừng. Không nên skip vô hạn.

Để log tài khoản lỗi, mình dùng `SkipListener`. Listener này bắt được item bị skip ở giai đoạn read, process hoặc write. Với tài khoản lỗi dữ liệu, thường lỗi xảy ra ở processor. Mình sẽ ghi vào bảng `batch_error_log` các thông tin như `jobExecutionId`, `stepExecutionId`, `accountNo` đã mask nếu cần, `customerId` đã mask, `reason`, `exception message`, `raw data reference`, `timestamp`. Không nên chỉ ghi file log, vì vận hành và đối soát cần query được danh sách tài khoản lỗi.

Ví dụ:

```

@Component
public class InterestSkipListener implements SkipListener<SavingAccount, InterestPos> {

    private final BatchErrorLogRepository errorLogRepository;

    public InterestSkipListener(BatchErrorLogRepository errorLogRepository) {
        this.errorLogRepository = errorLogRepository;
    }

    @Override
    public void onSkipInProcess(SavingAccount item, Throwable t) {
        BatchErrorLog log = new BatchErrorLog();
        log.setAccountNo(mask(item.getAccountNo()));
        log.setErrorReason(t.getMessage());
        log.setErrorType(t.getClass().getSimpleName());
        log.setCreatedAt(LocalDate.now());

        errorLogRepository.save(log);
    }
}

```

Tuy nhiên, trong thực tế cần cẩn thận: nếu ghi error log bằng cùng transaction của chunk đang rollback, log lỗi có thể bị rollback theo. Vì vậy nhiều hệ thống sẽ ghi error log bằng transaction riêng, hoặc dùng writer riêng, hoặc lưu vào một cơ chế bền vững độc lập. Nếu không để ý điểm này, tưởng đã log lỗi nhưng cuối cùng bảng error rỗng.

Ngoài skip, Spring Batch còn có retry. Retry phù hợp với lỗi tạm thời, ví dụ lock timeout, network chậm chờn, gọi service nội bộ timeout nhẹ. Nhưng với lỗi dữ liệu như interestRate null, retry 3 lần cũng không giải quyết được gì. Senior phải phân biệt rõ lỗi transient thì retry, lỗi data thì skip, lỗi hệ thống nghiêm trọng thì fail.

Một điểm nữa trong batch banking là restartability. Nếu job fail ở giữa vì lỗi hệ thống, mình không muốn chạy lại từ đầu và tính lãi trùng. Spring Batch có JobRepository lưu execution context, nhưng logic writer cũng phải idempotent. Ví dụ ghi interest posting phải có unique key theo accountNo + interestDate + jobId hoặc businessDate để tránh ghi trùng khi restart.

Tóm lại, mình sẽ dùng chunk-oriented step, bật fault tolerance, cấu hình skip cho lỗi dữ liệu có kiểm soát, đặt skipLimit, dùng SkipListener để log tài khoản lỗi vào bảng riêng, phân biệt retry và skip, đồng thời đảm bảo writer idempotent để job có thể restart an toàn. Với banking batch, mục tiêu không phải "cố chạy hết bằng mọi giá", mà là "chạy tiếp với lỗi cục bộ nhưng dừng ngay khi có dấu hiệu lỗi hệ thống hoặc lỗi diện rộng".

Ghi chú thuật ngữ:

Spring Batch là framework xử lý batch job trong Spring.

Chunk-oriented processing là cách xử lý theo từng nhóm item: đọc, xử lý, ghi theo chunk.

Fault-tolerant là khả năng chịu lỗi cục bộ mà không làm chết toàn bộ job.

Skip là bỏ qua item lỗi theo rule đã cấu hình.

SkipListener là listener dùng để bắt và xử lý item bị skip.

Retry là thử lại khi gặp lỗi tạm thời.

Restartability là khả năng chạy tiếp job từ điểm đã lưu sau khi fail.

12. Vì sao Microservices ngân hàng ưu tiên Saga Pattern thay vì Two-Phase Commit cho giao dịch phân tán? Ví dụ luồng mua vé máy bay/thẻ cào trên App.

Trong kiến trúc microservices, mỗi service thường sở hữu database riêng. Ví dụ Payment Service quản lý giao dịch thanh toán, Account Service quản lý số dư, Loyalty Service quản lý điểm thưởng, Notification Service gửi thông báo, Partner Service gọi nhà cung cấp vé máy bay hoặc thẻ cào. Khi một nghiệp vụ cần đi qua nhiều service, câu hỏi là làm sao đảm bảo tính nhất quán nếu một bước giữa chừng thất bại.

Two-Phase Commit, hay 2PC, là một cơ chế transaction phân tán kiểu cổ điển. Nó có một coordinator điều phối các resource. Phase đầu hỏi tất cả resource "có chuẩn bị commit được không?". Nếu tất cả đồng ý, phase hai mới yêu cầu commit. Nghe thì rất

đảm bảo consistency, nhưng trong microservices banking hiện đại, 2PC thường không được ưu tiên vì nó làm các service bị coupling chặt, giữ lock lâu, giảm availability, khó scale, và rất khó áp dụng khi một bên là hệ thống ngoài như ví điện tử, cổng thanh toán, airline, telco, hoặc core banking không hỗ trợ XA transaction.

Trong giao dịch online, giữ lock phân tán lâu là cực kỳ nguy hiểm. Ví dụ gọi sang nhà cung cấp vé máy bay có thể mất vài giây, thậm chí timeout. Nếu dùng 2PC và giữ transaction/lock trong thời gian đó, hệ thống tài khoản có thể bị nghẽn. Ngoài ra, trong microservices, mỗi service có thể dùng database khác nhau, message broker khác nhau, thậm chí có service bên ngoài không thuộc quyền kiểm soát. Không thể bắt tất cả tham gia cùng một global transaction.

Saga Pattern giải quyết theo hướng khác. Thay vì một transaction phân tán lớn, saga chia nghiệp vụ thành nhiều local transaction nhỏ. Mỗi local transaction commit trong database riêng của service đó. Nếu một bước sau thất bại, hệ thống chạy các hành động bù trừ, gọi là compensation, để đưa nghiệp vụ về trạng thái chấp nhận được.

Ví dụ luồng mua thẻ cào trên Mobile App có thể như sau. Người dùng chọn mệnh giá 100.000 VND. Payment Service tạo transaction ở trạng thái INIT. Account Service giữ tiền hoặc trừ tiền tài khoản, transaction chuyển sang DEBITED. Partner Service gọi nhà cung cấp thẻ cào. Nếu nhà cung cấp trả mã thẻ thành công, Payment Service chuyển trạng thái SUCCESS, lưu mã thẻ, gửi notification. Nếu nhà cung cấp timeout hoặc trả lỗi chắc chắn thất bại, hệ thống gọi compensation để hoàn tiền hoặc release số tiền đã giữ, rồi chuyển giao dịch sang FAILED hoặc REVERSED.

Ở đây, mỗi bước đều là local transaction. Debit tiền là một transaction trong Account Service. Gọi partner và lưu kết quả là transaction trong Payment/Partner Service. Hoàn tiền là một transaction bù trừ khác. Hệ thống không cần giữ một transaction database mở xuyên suốt toàn bộ thời gian gọi partner.

Với mua vé máy bay, saga còn rõ hơn. Luồng có thể gồm kiểm tra giá vé, giữ chỗ, trừ tiền, xuất vé, gửi email. Nếu giữ chỗ thành công nhưng trừ tiền thất bại, phải hủy giữ chỗ. Nếu trừ tiền thành công nhưng xuất vé thất bại, phải hoàn tiền hoặc chuyển sang trạng thái pending để vận hành xử lý với hãng. Không phải lúc nào compensation cũng đưa hệ thống về đúng trạng thái ban đầu ngay lập tức. Trong banking, nhiều trường hợp là eventual consistency: giao dịch có thể pending một thời gian rồi được đối soát, reverse hoặc manual handling.

Saga có hai kiểu triển khai chính là orchestration và choreography. Với orchestration, có một orchestrator điều phối từng bước, ví dụ Payment Orchestrator gọi Account Service, Partner Service, Notification Service. Cách này dễ kiểm soát flow, dễ audit, phù hợp với nghiệp vụ banking phức tạp. Với choreography, các service publish event và service

khác tự phản ứng, ví dụ PaymentCreated, AccountDebited, VoucherIssued. Cách này giảm coupling trực tiếp nhưng khó theo dõi flow nếu hệ thống lớn. Trong banking, mình thường thiên về orchestration cho luồng tiền vì cần kiểm soát trạng thái rõ ràng, audit dễ và xử lý lỗi minh bạch.

Một điểm rất quan trọng là Saga không có nghĩa là dữ liệu luôn consistent ngay lập tức. Nó chấp nhận eventual consistency. Vì vậy phải có transaction state machine rõ ràng: INIT, PROCESSING, DEBITED, PARTNER_PENDING, SUCCESS, FAILED, REVERSING, REVERSED. Không nên chỉ có boolean success/fail. Hệ thống thanh toán thực tế sống nhờ trạng thái trung gian, retry job, reconciliation job và dashboard vận hành.

Tóm lại, microservices banking ưu tiên Saga hơn 2PC vì Saga phù hợp với hệ thống phân tán, scale tốt hơn, không giữ lock lâu, làm việc được với service/database độc lập và đối tác bên ngoài. 2PC cho consistency mạnh hơn nhưng đánh đổi availability, performance và coupling. Với luồng thanh toán dịch vụ, Saga cho phép commit từng bước và bù trừ khi bước sau thất bại, miễn là hệ thống có idempotency, trạng thái giao dịch rõ ràng và cơ chế đối soát tốt.

Ghi chú thuật ngữ:

Saga Pattern là mẫu thiết kế chia giao dịch phân tán thành nhiều local transaction kèm bước bù trừ.

Two-Phase Commit là cơ chế commit phân tán qua hai pha prepare và commit.

Compensation là hành động bù trừ, ví dụ hoàn tiền sau khi bước sau thất bại.

Eventual consistency là nhất quán cuối cùng, dữ liệu có thể lệch tạm thời nhưng sẽ được đồng bộ về sau.

Orchestration là mô hình có một service điều phối toàn bộ flow.

Choreography là mô hình các service phản ứng với event mà không có điều phối trung tâm.

XA transaction là chuẩn transaction phân tán giữa nhiều resource như nhiều database.

13. Cao điểm lễ Tết request tăng đột biến. Triển khai Rate Limiting bằng thuật toán nào và đặt ở API Gateway hay Application?

Trong các đợt cao điểm như lễ Tết, lương thưởng, Black Friday, ngày mở bán vé, hoặc thời điểm thanh toán hóa đơn cuối tháng, request có thể tăng rất mạnh. Nếu không có rate limiting, hệ thống downstream như Account Service, Payment Service, Core Banking, OTP Service, Notification Service có thể bị quá tải. Rate limiting không chỉ để chống abuse, mà còn là cơ chế bảo vệ năng lực xử lý hữu hạn của hệ thống.

Thuật toán mình hay ưu tiên cho API Gateway là Token Bucket hoặc Leaky Bucket, tùy mục tiêu. Token Bucket cho phép burst trong giới hạn. Ví dụ một user bình thường được 10 request/giây, bucket chứa tối đa 20 token. Nếu user im lặng một lúc, token tích lũy, sau đó họ có thể gửi một burst nhỏ mà vẫn được chấp nhận. Điều này phù hợp với mobile app, vì request có thể đến theo cụm khi user mở app hoặc màn hình reload.

Leaky Bucket thì làm mượt traffic hơn, request đi ra với tốc độ ổn định. Nó phù hợp nếu downstream rất nhạy với burst. Tuy nhiên với API user-facing, Token Bucket thường linh hoạt hơn vì không bóp quá cứng các burst hợp lệ ngắn hạn.

Ngoài ra có Fixed Window và Sliding Window. Fixed Window dễ implement nhưng có lỗi biên cửa sổ. Ví dụ giới hạn 100 request/phút, user có thể gửi 100 request ở giây cuối phút trước và 100 request ở giây đầu phút sau, tạo burst 200 request trong vài giây. Sliding Window chính xác hơn nhưng tốn chi phí lưu trạng thái hơn. Trong production, nhiều API Gateway hoặc Redis-based limiter dùng biến thể sliding window log/counter hoặc token bucket.

Mình thường đặt rate limiting chính ở API Gateway. Lý do là Gateway là cửa vào hệ thống, chặn sớm sẽ tiết kiệm tài nguyên cho application phía sau. Nếu request quá hạn mức mà vẫn đi vào service rồi mới bị reject, service vẫn mất CPU, thread, connection, log, tracing. Gateway cũng là nơi để áp dụng rule theo clientId, userId, IP, deviceId, API key, partnerId, endpoint và channel.

Ví dụ với Mobile App, có thể limit theo userId hoặc deviceId cho các API nhạy cảm như login, OTP, balance inquiry, payment initiation. Với đối tác ví điện tử, limit theo partnerId + endpoint. Với public API, limit theo IP/API key. Với nội bộ, limit theo service identity.

Tuy nhiên, chỉ Gateway là chưa đủ. Application vẫn nên có rate limiting hoặc concurrency limiting cục bộ cho các nghiệp vụ đặc biệt quan trọng. Ví dụ Payment Service có thể giới hạn số giao dịch đang xử lý đồng thời cho một partner, hoặc giới hạn số request debit vào Account Service. Đây là lớp bảo vệ thứ hai, vì không phải traffic nào cũng đi qua cùng một gateway, và đôi khi internal retry/event cũng có thể gây quá tải.

Trong banking, rate limiting phải đi cùng priority. Không phải API nào cũng quan trọng như nhau. Truy vấn banner, danh sách khuyến mãi, lịch sử thông báo có thể bị limit mạnh hoặc degrade. Nhưng API xác thực, thanh toán, truy vấn số dư, xác nhận giao dịch cần chính sách riêng. Với khách hàng VIP, merchant lớn hoặc đối tác chiến lược, hạn mức cũng có thể khác. Nhưng mọi exception phải được kiểm soát, không được mở vô hạn.

Khi reject do rate limit, response nên rõ ràng, thường là HTTP 429, có thể kèm Retry-After. Mobile app cần hiểu để không retry điên cuồng. Nếu client bị 429 mà lập tức

retry liên tục, rate limit lại tạo thêm traffic. Vì vậy cần phối hợp với client exponential backoff và thông báo người dùng phù hợp.

Tóm lại, mình sẽ ưu tiên Token Bucket ở API Gateway để chặn burst hợp lệ nhưng vẫn bảo vệ hệ thống. Với các downstream quan trọng, bổ sung application-level limit hoặc bulkhead. Không nên đặt rate limiting chỉ ở application vì chặn quá muộn, cũng không nên chỉ đặt ở gateway vì có thể thiếu bảo vệ theo logic nghiệp vụ. Thiết kế tốt là nhiều lớp: Gateway limit theo identity/endpoint, Application limit theo business resource và downstream capacity.

Ghi chú thuật ngữ:

Rate Limiting là giới hạn tần suất request trong một khoảng thời gian.

Token Bucket là thuật toán dùng token để cho phép request, có thể chịu burst nhỏ.

Leaky Bucket là thuật toán làm mượt request theo tốc độ cố định.

Fixed Window là giới hạn theo cửa sổ thời gian cố định.

Sliding Window là giới hạn theo cửa sổ trượt, chính xác hơn fixed window.

HTTP 429 là mã phản hồi Too Many Requests.

Exponential backoff là chiến lược retry với thời gian chờ tăng dần.

14. Hai giao dịch cùng rút tiền từ một tài khoản dễ gây Race Condition. Phân biệt Pessimistic Locking và Optimistic Locking. Với luồng thanh toán QR Code, ưu tiên khóa nào để cân bằng hiệu năng và an toàn?

Race condition trong rút tiền là một lỗi rất nguy hiểm. Ví dụ tài khoản có 1.000.000 VND. Hai request rút 800.000 VND đến gần như cùng lúc. Nếu cả hai cùng đọc số dư là 1.000.000, cả hai đều thấy đủ tiền, rồi cùng trừ, hệ thống có thể cho phép chi vượt số dư. Đây là lỗi kinh điển của concurrent transaction.

Pessimistic Locking là cách tiếp cận "tôi giả định sẽ có xung đột, nên tôi khóa trước". Khi một transaction đọc tài khoản để trừ tiền, nó khóa row tài khoản đó bằng cơ chế như `SELECT ... FOR UPDATE`. Transaction khác muốn cập nhật cùng row phải chờ transaction đầu tiên commit hoặc rollback. Cách này an toàn và dễ hiểu, phù hợp với nghiệp vụ tiền vì đảm bảo tại một thời điểm chỉ một transaction được thay đổi số dư của tài khoản đó.

Ví dụ với JPA:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select a from Account a where a.accountNo = :accountNo")
```

```
Account findForUpdate(@Param("accountNo") String accountNo);
```

Hoặc SQL:

```
SELECT * FROM account  
WHERE account_no = ?  
FOR UPDATE;
```

Nhược điểm của pessimistic locking là có thể làm giảm throughput nếu nhiều giao dịch cùng đánh vào một tài khoản. Nó cũng có nguy cơ lock wait timeout hoặc deadlock nếu thứ tự khóa không nhất quán. Ví dụ một transaction chuyển tiền từ A sang B, transaction khác chuyển từ B sang A, nếu mỗi bên khóa một tài khoản rồi chờ tài khoản còn lại, có thể deadlock. Vì vậy khi khóa nhiều tài khoản, phải có quy tắc thứ tự khóa ổn định, ví dụ luôn khóa theo accountNo tăng dần.

Optimistic Locking thì ngược lại. Nó giả định xung đột không thường xuyên, nên không khóa row khi đọc. Mỗi record có một cột version. Khi update, câu SQL kiểm tra version cũ. Nếu version không còn đúng, nghĩa là có transaction khác đã cập nhật trước, update thất bại và ứng dụng phải retry hoặc trả lỗi.

Ví dụ entity:

```
@Entity  
public class Account {  
  
    @Id  
    private Long id;  
  
    private BigDecimal balance;  
  
    @Version  
    private Long version;  
}
```

Khi hai transaction cùng đọc version 10, transaction đầu update thành version 11 thành công. Transaction thứ hai update với điều kiện version 10 sẽ fail vì version hiện tại đã là 11. Cách này không giữ lock lâu, hiệu năng tốt khi conflict thấp. Nhưng khi conflict cao, retry nhiều có thể làm latency tăng và gây áp lực hệ thống.

Với luồng thanh toán QR Code hiện nay, mình sẽ phân biệt rõ phần nào cần khóa mạnh, phần nào có thể optimistic. Nếu là bước thực sự ghi nợ tài khoản khách hàng, mình ưu tiên an toàn dữ liệu trước. Có hai cách thực tế mình hay dùng.

Cách thứ nhất là dùng atomic conditional update ở database, ví dụ:

```
UPDATE account
SET balance = balance - :amount
WHERE account_no = :accountNo
AND balance >= :amount;
```

Sau đó kiểm tra số row affected. Nếu row affected = 1, debit thành công. Nếu = 0, không đủ tiền hoặc tài khoản không hợp lệ. Cách này rất hiệu quả vì không cần read rồi update tách rời, giảm cửa sổ race condition. Đây là một dạng tận dụng atomicity của database, thường rất phù hợp cho debit đơn tài khoản.

Cách thứ hai là dùng pessimistic lock cho các luồng phức tạp cần đọc nhiều thông tin, validate nhiều rule, cập nhật nhiều bảng ledger liên quan. Ví dụ phải kiểm tra số dư khả dụng, hạn mức ngày, hạn mức merchant, trạng thái phong tỏa, số tiền đang hold, fee, cashback, thì mình có thể khóa account balance row trong phạm vi transaction ngắn nhất có thể. Điều quan trọng là không được giữ lock trong lúc gọi API bên ngoài như QR provider, merchant, notification hoặc risk engine chậm. Chỉ giữ lock khi thao tác dữ liệu cốt lõi trong database.

Với QR Code payment, traffic có thể cao, nhưng conflict trên cùng một tài khoản thường không quá lớn trừ khi user bấm nhiều lần hoặc merchant quét nhiều giao dịch liên tục. Tuy nhiên hậu quả overdraft là nghiêm trọng. Vì vậy mình sẽ ưu tiên thiết kế debit bằng atomic conditional update hoặc pessimistic lock ngắn trong Account Service. Optimistic locking có thể dùng cho các dữ liệu ít nhạy cảm hơn hoặc conflict thấp hơn, ví dụ cập nhật metadata giao dịch, trạng thái hiển thị, profile, hoặc một số aggregate không trực tiếp quyết định số dư.

Nếu dùng optimistic locking cho debit, vẫn có thể an toàn nếu retry được kiểm soát. Nhưng trong thanh toán realtime, retry do version conflict cần rất cẩn thận. Nếu retry đọc lại số dư và trừ lại, phải đảm bảo cùng transaction id không bị xử lý hai lần. Do đó optimistic lock phải đi cùng idempotency key và ledger unique constraint. Nếu không, retry có thể tạo double posting.

Một thiết kế banking tốt thường không chỉ update balance đơn thuần. Nó có ledger entry bất biến. Balance có thể là số dư hiện tại, nhưng mọi thay đổi phải có bút toán debit/credit riêng, transaction reference duy nhất, và unique constraint để không ghi trùng. Như vậy dù dùng locking nào, hệ thống vẫn có audit trail và khả năng đối soát.

Tóm lại, pessimistic locking khóa trước, an toàn hơn khi conflict cao hoặc dữ liệu cực kỳ nhạy cảm nhưng có thể giảm throughput. Optimistic locking không khóa trước, hiệu

năng tốt khi conflict thấp nhưng phải xử lý retry/version conflict. Với QR Code payment, mình ưu tiên atomic conditional update hoặc pessimistic lock rất ngắn cho bước debit số dư, kết hợp idempotency và ledger unique constraint. Đây là điểm cân bằng tốt giữa hiệu năng và an toàn tiền.

Ghi chú thuật ngữ:

Race Condition là lỗi xảy ra khi nhiều luồng/giao dịch truy cập và thay đổi cùng dữ liệu theo thứ tự không kiểm soát.

Pessimistic Locking là khóa dữ liệu trước khi cập nhật vì giả định có thể xảy ra xung đột.

Optimistic Locking là không khóa trước, nhưng kiểm tra version khi update để phát hiện xung đột.

SELECT FOR UPDATE là câu SQL khóa row được đọc để transaction khác không sửa cùng lúc.

Version column là cột dùng để phát hiện record đã bị thay đổi bởi transaction khác.

Atomic update là thao tác update diễn ra như một đơn vị không thể bị chen ngang.

Ledger là sổ ghi bút toán giao dịch, dùng cho audit và đối soát.



bạn có thể tạo ra các hình ảnh để mô tả trực quan hơn cho các câu trả lời trên được không

Đã ngừng suy nghĩ >



Chỉnh sửa



